符也不上报事件，直接丢掉。

evdev 队列：每个读者一份。 /dev/input/eventN 的读语义按 Linux evdev 实现，一共四点：

- **minor 稳定绑定设备。** input_register_device() 从设备表里取一个空槽，槽号就是 eventN；open() 用 inode 的 minor 反查这张表，因此不会出现所有设备都“晋升”成 event0。devtmpfs 预先创建 /dev/input/event0..event3（主设备号 INPUT = 4，minor 0 ~ 3），空槽的节点还在但 open() 失败——这正是 Linux 的 ENODEV 语义，好处是不动态增删节点时也不会把旧节点错指给后来注册的设备。
- **每个 open 一个客户端。** evdev_open() 分配一个 evdev_client（EVDEV_BUFFER = 256 条事件的环形缓冲）挂到 dev->clients，两个读者各读各的，不会互相偷事件。
- **只暴露完整帧。** packet_head 标记最后一个完整帧的末尾，read() 最多读到那里；溢出时把尾巴丢掉并插入 SYN_DROPPED，读取者据此丢弃自己缓存的按键状态，等下一个 SYN_REPORT 再恢复。
- **断开是有顺序的。** input_unregister_device() 先把设备从表里摘掉、唤醒阻塞的读者，再让旧 fd 读完已经提交的完整帧，之后 read() 才返回错误。

每条事件都是一个 struct input_event：uint16 type、uint16 code、int value，read() 一次可以拿走多条。键盘发布 EV_KEY（value 1 按下、0 松开），Boot Mouse 还会发布 BTN_LEFT/BTN_RIGHT/BTN_MIDDLE 和 EV_REL 的 REL_X/REL_Y/REL_WHEEL，末尾统一跟一个 EV_SYN/SYN_REPORT。在真机上可以直接验证：

```
$ ls /dev/input
event0  event1  event2  event3
$ cat /proc/devices        # Character devices: 4 input/event0
$ evtest /dev/input/event0 # 每次按下/松开各一条 EV_KEY，帧尾一条 EV_SYN
```

引用计数与生命周期。 input_dev 由 kalloc() 分配，初始引用属于类驱动；每个打开的 event fd 再加一个引用。usb_free_urb() 和 kfree(kbd) 发生在 remove 路径上，而 input_dev 的内存要等最后一个 fd 关闭才释放——这样“驱动已经卸载”和“用户还在读旧事件”两件事可以同时成立。注销路径因此不会睡眠等待用户空间关闭 /dev/input/event0。

7.4.7 Hub 中断驱动的 USB 热插拔

键盘按键报告和 USB 拓扑变化是两种不同的事件。按键数据由键盘自己的 interrupt-IN URB 接收；热插拔则由 LAN951x Hub 的 interrupt-IN endpoint 通知主机。当前 DWC2 为 Hub 的长期 URB 固定使用 channel 7，为唯一 HID 数据 URB 使用 channel 6。Hub 空闲时会对 interrupt-IN 返回 NAK，HCD 按 bInterval 重新调度请求；只有端口变化位图非零时才唤醒 Hub 事件 worker，不再每隔固定时间轮询所有端口的 EP0。

```
Hub interrupt endpoint
  -> DWC2 channel 7 DMA receives change bitmap
  -> IRQ 9 records completion and queues hub_event_work
  -> dwc2_hub_wq examines changed ports only
  -> GET_STATUS / CLEAR_FEATURE for those ports
  -> compare connected bitmap
```

硬中断只处理 host-channel 完成并排入工作项，不在 IRQ 中进行 Hub control 传输。独立的 dwc2_hub_wq 根据 bitmap 中标记的端口发送 GET_STATUS，读取连接状态和变化位，再用 CLEAR_FEATURE 清除已处理的 port change feature。比较新旧 connected bitmap 后，只有拓扑确实变化才启动重枚举；因此空闲 NAK 不会造成无意义的全端口扫描。

设备拔出时，类驱动和 input/evdev 的资源按逆序回收：

```
usb_device_unregister(old_child)
```